SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	P. Sai Charan Tej	**Reg. No.:**	192465048
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q43

SECTION A – SCENARIO BASED ASSESSMENT

Q43. A student database stores records unsorted due to continuous updates. The system retrieves student details using sequential search.

a) Explain how Linear Search works here.

b) Analyze its efficiency in different cases.

c) Justify whether sorting would help.

Code of Conduct

I P. Sai Charan Tej(192465048)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q43. A student database stores records unsorted due to continuous updates. The system retrieves student details using sequential search.

Explanation

Linear Search (Sequential Search) examines each student record one by one until the required record is found or all records have been checked. It is suitable for unsorted databases where records are frequently inserted or updated.

a) Explain how Linear Search works here.

Since the student records are **unsorted**, Linear Search checks each record one by one from the beginning until the required student record is found or all records have been examined.

Execution

Student Records (Roll Numbers):

Index	Roll No.
0	101
1	105
2	103
3	104
4	102
5	106

Search Key = 104

Step	Compared Value	Result
1	101	Not Found
2	105	Not Found
3	103	Not Found
4	104	**Found**

Output:

Student with Roll No. 104 found at Index 3.

Total Comparisons = 4

a) Explain how Linear Search works here.

Python 3 Program:

```
roll_no = [101, 105, 103, 104, 102, 106]
key = 104
comparisons = 0

for i in range(len(roll_no)):
    comparisons += 1
    print("Comparing", roll_no[i], "with", key)
    if roll_no[i] == key:
        print("Student Found at Index:", i)
        break

print("Total Comparisons:", comparisons)
```

Execution

Input:

Student Records = [101, 105, 103, 104, 102, 106]

Search Key = 104

Output:

Comparing 101 with 104

Comparing 105 with 104

Comparing 103 with 104

Comparing 104 with 104

Student Found at Index: 3

Total Comparisons: 4

b) Analyze its efficiency in different cases.

Best Case

- The required student is at the **first position**.
- Comparisons = 1
- **Time Complexity = O(1)**

Average Case

- The student is located somewhere in the middle.
- Comparisons ≈ n/2
- **Time Complexity = O(n)**

Worst Case

- The student is at the last position or not present.
- Comparisons = n
- **Time Complexity = O(n)**

Case	Comparisons	Time Complexity
Best	1	O(1)
Average	n/2	O(n)
Worst	n	O(n)

b) Analyze its efficiency in different cases.

Python 3 Program:

```
n = 6

print("Best Case    : O(1)")
print("Average Case : O(n)")
print("Worst Case   : O(n)")
```

Execution

Output:

Best Case : O(1)

Average Case : O(n)

Worst Case : O(n)

Best Case Comparisons = 1

Average Case Comparisons ≈ 3

Worst Case Comparisons = 6

c) Justify whether sorting would help.

Sorting the student records allows faster searching methods such as **Binary Search**, which has a time complexity of **O(log n)**.

However, in this database, records are **continuously updated** by frequent insertions and deletions. Maintaining the records in sorted order after every update requires additional time.

Therefore:

- **If updates are frequent:** Keeping the database unsorted and using **Linear Search** is more practical.
- **If searches are frequent and updates are rare:** Sorting the records and using **Binary Search** is a better choice because searching becomes much faster.

c) Justify whether sorting would help.

Python 3 Program:

```
records = [101,105,103,104,102,106]

print("Unsorted:", records)
records.sort()
print("Sorted:", records)

print("Linear Search : O(n)")
```

```
print("Binary Search : O(log n)")
```

Execution

Output:

Unsorted: [101, 105, 103, 104, 102, 106]

Sorted: [101, 102, 103, 104, 105, 106]

Linear Search : O(n)

Binary Search : O(log n)

Result / Conclusion

The student database is continuously updated, so keeping it sorted after every insertion or deletion is costly. Therefore, Linear Search is a practical choice for unsorted data. If updates are infrequent and searches are frequent, sorting the records and using Binary Search provides faster searching.